Module 3：Diffusion Transformer 数学原理与代码推导

本文沿用 DDPM 讲义的记号，从“扩散模型需要一个什么函数”开始，推到 DiT 的 patch、Transformer、adaLN-Zero、完整训练目标与 classifier-free guidance。仓库中的轻量实验可在 MNIST、FashionMNIST 或 CIFAR-10 像素上运行；原始论文设置是 ImageNet 图像经 VAE 压缩后的 latent。

配套材料：

- 学习计划：6_dit-case-study.md
- 一手资料：dit-primary-source-notes.md
- 教学代码：`code/image_dit/dit.py`、`code/image_dit/diffusion.py`
- 原始论文：Scalable Diffusion Models with Transformers
- 官方实现：facebookresearch/DiT

1. 最重要的分层：diffusion 与 denoiser

DDPM 定义 forward noising process：

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, \beta_t I), \qquad \alpha_t = 1 - \beta_t.$$

令 $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$，可以一步采样任意噪声时刻：

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

原始 DiT 训练一个噪声预测器：

$$\epsilon_\theta(x_t, t, c),$$

其中 c 是类别、文本或其他条件。简化目标为：

$$L_{simple} = \mathbb{E}_{x_0,t,\epsilon,c}\left[\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2\right].$$

从 UNet 换成 DiT 时，上面四行数学不变。改变的只是实现 ϵ_θ 的神经网络：

$$\mathrm{UNet}(x_t, t, c) \quad \longrightarrow \quad \mathrm{DiT}(x_t, t, c).$$

这是理解 DiT 的第一原则：**DiT 是 denoiser backbone，不等于一套新的 diffusion 概率过程**。同一个 noise schedule、loss 和 sampler 可以连接不同 backbone；第 9 节会在原始 epsilon target 之外加入 DDPM v-prediction 消融，但不会改变这个分层。

2. Pixel diffusion 与 latent diffusion

原始 DiT 不直接对 $256 \times 256 \times 3$ 像素做 Transformer。它继承 Latent Diffusion 的做法，用预训练 VAE encoder E 压缩图像：

$$z_0 = E(x_0), \qquad z_0 \in \mathbb{R}^{4\times 32\times 32} \quad (256\times 256 \text{ 输入的典型形状}).$$

扩散发生在 latent 上：

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1-\bar{\alpha}_t}\epsilon.$$

DiT 预测 latent noise，最后由 decoder D 还原图像：

$$\hat{x}_0 = D(\hat{z}_0).$$

VAE 压缩的作用不只是省显存。若空间边长缩小 8 倍，二维位置数量缩小 64 倍，而 self-attention 的成对交互数量近似再平方缩小。代价是生成器只能在 VAE 保留的信息空间里工作，VAE reconstruction 上限会影响最终细节。

本仓库轻量教学实验直接在 MNIST/FashionMNIST 的 $28\times 28\times 1$ 或 CIFAR-10 的 $32\times 32\times 3$ pixel space 工作，省去预训练 VAE 依赖。它用于验证架构与 diffusion recipe，不用于复现 ImageNet 指标。

3. 从图像到 patch token

设输入为：

$$x \in \mathbb{R}^{B\times C\times H\times W}.$$

用不重叠的 $p\times p$ patch 切分。网格尺寸与 token 数是：

$$H_p = H/p, \qquad W_p = W/p, \qquad N = H_pW_p.$$

每个 patch 展平后有 p^2C 个数，再线性投影到 hidden size d：

$$u_i = W_p\,\mathrm{vec}(x^{(i)}) + b_p, \qquad u_i \in \mathbb{R}^d.$$

代码通常用 kernel size 与 stride 都等于 p 的 `Conv2d` 一次完成切块和线性投影：

```
nn.Conv2d(C, d, kernel_size=p, stride=p)
```

卷积输出形状为 $[B, d, H_p, W_p]$，flatten 与 transpose 后为：

$$U \in \mathbb{R}^{B\times N\times d}.$$

3.1 两个具体例子

CIFAR-10 教学版：

$$H = W = 32,\ p = 4 \Longrightarrow H_p = W_p = 8,\ N = 64.$$

ImageNet-256 的 latent DiT-XL/2：VAE 后 $H = W = 32,\ p = 2$：

$$H_p = W_p = 16,\ N = 256.$$

把 patch 从 4 减到 2，token 数增加 4 倍；attention score matrix 从 N^2 增加到 16 倍。patch size 是 DiT 最强也最昂贵的扩展旋钮之一。

4. 二维位置编码

self-attention 本身对 token 排列没有二维空间概念。DiT 给每个 patch token 加固定二维 sine/cosine position embedding：

$$h_i^{(0)} = u_i + p_i.$$

二维编码可把行坐标与列坐标分别编码，再拼接：

$$p_{(r,c)} = [\mathrm{PE}(r); \mathrm{PE}(c)].$$

单个一维坐标 r 的频率特征为：

$$\mathrm{PE}(r)_{2k} = \sin(r\omega_k), \qquad \mathrm{PE}(r)_{2k+1} = \cos(r\omega_k),$$

$$\omega_k = 10000^{-k/m}.$$

固定位置编码没有可学习参数，且不同分辨率可重新生成。它不是唯一选择；后续视频模型常用 3D/MM-RoPE 处理时间、宽、高和多模态位置。

5. 时间与类别条件

DiT 的每次 denoising forward 都必须知道当前噪声时刻 t。先得到 sinusoidal embedding：

$$e_t = [\cos(t\omega_0), \dots, \cos(t\omega_{m-1}), \sin(t\omega_0), \dots, \sin(t\omega_{m-1})],$$

再通过两层 MLP：

$$c_t = W_2\,\mathrm{SiLU}(W_1 e_t + b_1) + b_2.$$

类别 y 查 embedding table 得到 c_y。原始 class-conditional DiT 将二者相加：

$$c = c_t + c_y, \qquad c \in \mathbb{R}^{B\times d}.$$

这里的 c 不是额外图像 token，而是生成每层 LayerNorm modulation 参数的全局条件向量。对长文本，现代 MMDiT 常保留文本 token 并通过 cross/joint attention 交互，不能只靠一个 pooled vector 替代全部语义。

6. 从 LayerNorm 到 adaLN-Zero

6.1 LayerNorm

对一个 token $h \in \mathbb{R}^d$：

$$\mu(h) = \frac{1}{d}\sum_{j=1}^{d} h_j, \qquad \sigma^2(h) = \frac{1}{d}\sum_{j=1}^{d}(h_j - \mu)^2,$$

$$\mathrm{LN}(h) = \frac{h - \mu(h)}{\sqrt{\sigma^2(h) + \varepsilon}}.$$

普通 LayerNorm 再使用固定可学习 γ, β。adaptive LayerNorm 改由条件 c 生成 sample-dependent shift 和 scale：

$$\mathrm{adaLN}(h, c) = \mathrm{LN}(h) \odot (1 + s(c)) + b(c).$$

写成 $1 + s$ 而不是只乘 s，让 $s = 0, b = 0$ 时退化为普通无 affine 的 LayerNorm。

6.2 一个 DiT block 的六路参数

条件投影产生六个 d 维向量：

$$(b_{msa}, s_{msa}, g_{msa}, b_{mlp}, s_{mlp}, g_{mlp}) = W_c\,\mathrm{SiLU}(c).$$

attention residual branch：

$$\tilde{h} = h + g_{msa} \odot \mathrm{MSA}(\mathrm{adaLN}(h; b_{msa}, s_{msa})).$$

MLP residual branch：

$$h' = \tilde{h} + g_{mlp} \odot \mathrm{MLP}(\mathrm{adaLN}(\tilde{h}; b_{mlp}, s_{mlp})).$$

g 是 residual gate，b, s 控制进入分支前的特征。

6.3 Zero 的严格含义

将输出六路参数的线性层权重与 bias 初始化为零：

$$b = s = g = 0.$$

于是：

$$\tilde{h} = h + 0 \cdot \mathrm{MSA}(\mathrm{LN}(h)) = h,$$

$$h' = h + 0 \cdot \mathrm{MLP}(\mathrm{LN}(h)) = h.$$

因此每个 block 在初始化时是恒等映射。最终 projection 也零初始化，所以整个 DiT 初始输出为零。这个性质可以直接写成测试，不必凭训练曲线猜初始化是否正确。

注意：零输出不意味着没有梯度。第一步的 loss 会给最终线性层非零梯度；随着输出层离开零点，梯度再传入前面的 blocks。

7. Multi-Head Self-Attention 的计算

对 token matrix $H \in \mathbb{R}^{N \times d}$：

$$Q = HW_Q, \qquad K = HW_K, \qquad V = HW_V.$$

每个 head 维度 $d_h = d/n_h$：

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V.$$

多头输出拼接后再投影：

$$\mathrm{MSA}(H) = [head_1; \ldots; head_{n_h}]W_O.$$

attention score 为 $N \times N$，因此其主要 token 交互成本近似：

$$O(N^2 d).$$

MLP 的主要成本近似 $O(Nd^2r)$，r 是 expansion ratio。小 N、大 d 时 MLP/linear 可能主导；高分辨率或长视频导致 N 变大时，attention 更快成为瓶颈。因此只说“Transformer 是平方复杂度”仍不够，必须同时记录 $N, d, depth$。

8. Final layer 与 unpatchify

最后一层再次用条件调制：

$$O_i = W_o\,\mathrm{adaLN}(h_i, c) + b_o.$$

每个 token 输出 p^2C_{out} 个数：

$$O \in \mathbb{R}^{B\times N\times (p^2C_{out})}.$$

将 token index 拆成二维 patch 网格，将 token feature 拆成 patch 内的行、列、通道，再 permute/reshape：

$$[B, H_p, W_p, p, p, C_{out}] \longrightarrow [B, C_{out}, H_p p, W_p p].$$

若只预测均值参数，$C_{out} = C_{in}$。若同时学习 reverse variance，原始 DiT/官方训练配置默认 `learn_sigma=True`：

$$C_{out} = 2C_{in},$$

前一半通道是 mean parameterization（原论文为 epsilon；本仓库也提供 DDPM v-prediction 消融），后一半是 learned-range variance 的 raw values。完整版教学代码已经实现 mixed MSE/VB、learned variance、raw/rescaled VB 和 ancestral sampling；`minimal_dit.py` 则有意只保留 fixed-variance epsilon MSE，二者不能混为同一个能力边界。

9. 训练目标：模型如何连到 DDPM

先把常用简写固定下来：

$$a_t = \sqrt{\bar{\alpha}_t}, \qquad \sigma_t = \sqrt{1-\bar{\alpha}_t}, \qquad x_t = a_t x_0 + \sigma_t \epsilon.$$

注意这里的 σ_t 是 forward process 的噪声标准差，不是模型预测的 reverse variance。

9.1 epsilon prediction 与 DDPM v-prediction

原始 DiT 使用 epsilon prediction：

$$m_\theta(x_t, t, c) = \hat{\epsilon}_\theta, \qquad L_{\text{MSE}} = \mathbb{E}\left[\|\epsilon - \hat{\epsilon}_\theta\|_2^2\right].$$

由 forward 闭式可以反演 clean sample：

$$\hat{x}_0 = \frac{x_t - \sigma_t \hat{\epsilon}_\theta}{a_t}.$$

本仓库还提供同一个离散 DDPM probability path 上的 **v-parameterization** 消融（不少文献把这里的 a_t 直接记作 α_t）：

$$v = a_t \epsilon - \sigma_t x_0, \qquad L_v = \mathbb{E}\left[\|v - v_\theta(x_t, t, c)\|_2^2\right].$$

因为 $a_t^2 + \sigma_t^2 = 1$，(x_t, v) 是 (x_0, ϵ) 的二维旋转：

$$\begin{bmatrix} x_t \\ v \end{bmatrix} = \begin{bmatrix} a_t & \sigma_t \\ -\sigma_t & a_t \end{bmatrix} \begin{bmatrix} x_0 \\ \epsilon \end{bmatrix}.$$

逆变换不需要除以接近零的系数：

$$\hat{x}_0 = a_t x_t - \sigma_t \hat{v}_\theta, \qquad \hat{\epsilon} = \sigma_t x_t + a_t \hat{v}_\theta.$$

得到 $\hat{x}_0$ 或 $\hat{\epsilon}$ 后，后面的 DDPM posterior、learned variance 和 sampler 不变。**这不是原始 DiT 论文 recipe**：`paper_compatible` profile 固定使用 epsilon prediction；v-prediction 是为教学和小数据质量比较保留的显式消融。

9.2 learned-range reverse variance

forward posterior 为：

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I),$$

$$\tilde{\beta}_t = \beta_t \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t},$$

$$\tilde{\mu}_t = \frac{\beta_t \sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_t} x_0 + \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t} x_t.$$

固定方差模型直接令 reverse variance 等于 $\tilde{\beta}_t$。完整 DiT 则让第二个输出 head 给出 raw value r_θ，并采用 Improved DDPM 的 `LEARNED_RANGE`：

$$\ell_{\min,t} = \log \tilde{\beta}_t, \qquad \ell_{\max,t} = \log \beta_t, \qquad f_\theta = \frac{r_\theta + 1}{2},$$

$$\log \Sigma_\theta = (1-f_\theta)\ell_{\min,t} + f_\theta \ell_{\max,t}, \qquad \Sigma_\theta = \exp(\log \Sigma_\theta).$$

$t = 0$ 时 $\tilde{\beta}_0 = 0$，不能直接取对数；实现沿用 Improved DDPM，把 clipped posterior log variance 的首项替换为 $\log \tilde{\beta}_1$。另一个容易漏掉的细节是：官方实现并不先对 r_θ 做 `tanh` 或 clamp，所以 f_θ 名义上是插值系数，数值上也允许外推。

将 $\hat{x}_0$ 代入 posterior mean 后，模型 reverse transition 是：

$$p_\theta(x_{t-1} \mid x_t, c) = \mathcal{N}\left(\tilde{\mu}_t(x_t, \hat{x}_0), \Sigma_\theta(x_t, t, c)\right).$$

9.3 MSE + VB，以及 mean head 的 stop-gradient

若模型输出 $[m_\theta, r_\theta]$ 两组 C 通道，训练目标是：

$$L = L_{\text{MSE}} + w_{VB} L_{VB}.$$

MSE 训练 mean head：epsilon 模式以 ϵ 为 target，v 模式以 v 为 target。VB 项训练 variance head，但在构造 $\hat{x}_0$ 和 model mean 时对 m_θ 停止梯度：

$$\hat{x}_0^{VB} = \text{invert}(x_t, t, \text{stopgrad}(m_\theta)).$$

因此：

$$\frac{\partial L_{VB}}{\partial m_\theta} = 0, \qquad \frac{\partial L_{VB}}{\partial r_\theta} \neq 0.$$

这避免不稳定的 variational term 改写已经由 simple MSE 学习的 mean prediction，同时仍让 learned variance 得到梯度。这正是 Improved DDPM 与官方 DiT diffusion code 的 hybrid-loss 处理。

9.4 为什么 $t = 0$ 是 decoder NLL，而 $t > 0$ 是 KL

逐样本 VB 项按 timestep 分段：

$$L_{VB,t} = \begin{cases} -\log p_\theta(x_0 \mid x_1), & t = 0, \\ D_{KL}(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)), & t > 0. \end{cases}$$

当 $t > 0$ 时，两边都是 Gaussian，可以用 Gaussian KL 的闭式。对标量维度，若 $q = \mathcal{N}(\mu_q, e^{\ell_q})$、$p = \mathcal{N}(\mu_p, e^{\ell_p})$：

$$D_{KL}(q\|p) = \frac{1}{2}\left[-1 + \ell_p - \ell_q + e^{\ell_q - \ell_p} + (\mu_q - \mu_p)^2 e^{-\ell_p}\right].$$

$t = 0$ 已抵达观测数据，目标不再是另一个 latent transition，而是给离散像素赋概率；这里代码的零基 `t=0` 对应通常一基推导中的最后一步 $x_1 \to x_0$。对于缩放到 $[-1, 1]$ 的 8-bit 像素，内部 bin 的半宽为 $1/255$。若 model mean 为 μ、标准差为 s，则内部像素值 x 的概率质量是：

$$p(x \mid \mu, s) = \Phi\left(\frac{x + 1/255 - \mu}{s}\right) - \Phi\left(\frac{x - 1/255 - \mu}{s}\right).$$

边界 $x \approx -1$ 和 $x \approx 1$ 分别使用左尾 $\Phi(\cdot)$ 与右尾 $1 - \Phi(\cdot)$，避免把图像取值范围以外的 Gaussian 质量错误丢掉。实现对 channel 与空间维取平均、加负号，并除以 $\log 2$，所以 decoder NLL 和 KL 都以 bits/dim 计。

9.5 raw VB 与 rescaled VB

仓库把两种权重语义显式区分：

$$w_{VB} = \begin{cases} 1, & \text{raw VB / 官方 DiT 默认 MSE loss type,} \\ T/1000, & \text{rescaled VB / RESCALED_MSE.} \end{cases}$$

训练 schedule 恰为 $T = 1000$ 时二者数值相同，但语义仍不同；改变训练步数后不能再把它们当作同一 recipe。`paper_compatible` 使用 raw VB，`pixel_teaching` 保留 rescaled VB 作为明确的教学/legacy 选项。若只输出 C 通道、使用 fixed variance，则当前完整版退化为单独的 MSE，不计算 variance VB。

9.6 一次完整训练 step

1. 取 clean image/latent x_0 与 condition y；
2. 均匀采样 $t \in \{0, \dots, T-1\}$；
3. 采样 $\epsilon \sim \mathcal{N}(0, I)$；
4. 构造 $x_t = a_t x_0 + \sigma_t \epsilon$；
5. DiT 输出 mean parameterization，以及可选的 learned-range variance raw values；
6. 用 epsilon 或 v target 计算 MSE；若学习方差，再计算带 stop-gradient 的 VB；
7. 更新在线模型参数，然后更新 EMA 参数。

9.7 EMA：训练在线参数，使用平滑参数推理

设 optimizer 更新后的在线参数为 θ_k，EMA 参数为 $\bar{\theta}_k$，则每个 optimizer step 后执行：

$$\bar{\theta}_k = \rho \bar{\theta}_{k-1} + (1-\rho)\theta_k, \qquad 0 < \rho < 1.$$

原始 DiT recipe 取 $\rho = 0.9999$，并使用 EMA 权重生成论文样本。EMA 模型在训练开始时复制在线模型，但不参与反向传播；checkpoint 同时保存在线权重和 EMA 权重，才能既恢复训练又复现推理。EMA 不是 diffusion 方程或 DiT block，它是训练轨迹上的低通滤波：近期单步更新的权重只有 $1-\rho$，较早权重的影响按 ρ^j 衰减。

9.8 no-clip 与论文式 1000→250 respaced ancestral DDPM

从 model mean 采样一步：

$$x_{t-1} = \tilde{\mu}_t(x_t, \hat{x}_0) + \mathbf{1}_{t>0}\sqrt{\Sigma_\theta}\, z, \qquad z \sim \mathcal{N}(0, I).$$

$t = 0$ 时随机项严格为零，这是必须测试的边界条件。`pixel_teaching` 可在反演后执行 $\hat{x}_0 \leftarrow \mathrm{clip}(\hat{x}_0, -1, 1)$，作为像素实验的稳定化选项；官方 latent DiT 的 loss 和采样都传入 `clip_denoised=False`。VAE latent 并没有 $[-1,1]$ 的硬边界，clamp 会直接改变 $\tilde{\mu}_t$ 以及 variance head 接收到的 VB 信号。因此 `paper_compatible` 在训练 VB 与采样两处都使用 **no-clip**，不能只在最终显示图片时“取消裁剪”来冒充等价。

论文评估用 250-step ancestral DDPM，但模型仍在原始 $T = 1000$ 个 timestep 上训练。设从原 schedule 选择：

$$0 = s_0 < s_1 < \cdots < s_{K-1} = 999, \qquad K = 250,$$

其中 s_j 由等间隔 fractional stride 后取整得到。不能新建一条普通 250-step linear-beta schedule；应定义：

$$\beta'_j = 1 - \frac{\bar{\alpha}_{s_j}}{\bar{\alpha}_{s_{j-1}}}, \qquad \bar{\alpha}_{s_{-1}} := 1.$$

于是新 schedule 的累计乘积严格保留被选中的原始信噪比：

$$\bar{\alpha}'_j = \prod_{i=0}^{j} (1 - \beta'_i) = \bar{\alpha}_{s_j}.$$

采样器用 β'_j 计算跨越多个原始 timestep 的 posterior，但把 local index j 映射回 s_j 后送入 DiT 的 timestep embedding。每一步仍从 Gaussian reverse transition 加噪，所以这是 **respaced ancestral DDPM**，不是 DDIM。仓库另有快速 DDIM 供教学交互；它不等于论文的 250-step 指标 recipe。

10. Classifier-Free Guidance：all 与 first-3

训练时以概率 p_{drop} 把类别替换为 learned null class $\varnothing$。同一模型学到：

$$\epsilon_\theta(x_t, t, y) \quad 与 \quad \epsilon_\theta(x_t, t, \varnothing).$$

推理时必须让 conditional 与 unconditional 分支从同一个 x_t, t 出发。对原始 DiT 的 epsilon prediction：

$$\epsilon_{cfg} = \epsilon_{uncond} + s\big(\epsilon_{cond} - \epsilon_{uncond}\big).$$

边界：

$$s = 0 \Rightarrow \epsilon_{cfg} = \epsilon_{uncond},$$

$$s = 1 \Rightarrow \epsilon_{cfg} = \epsilon_{cond}.$$

$s > 1$ 外推条件方向，通常提高条件一致性但可能降低多样性、产生过饱和或伪影。CFG 不是训练 loss 的替代品，而是同时训练 cond/uncond 后的采样组合。若使用 DDPM v-prediction，仓库对 v_θ 应用同样的线性组合，再通过第 9.1 节的逆变换得到 epsilon 与 x_0；这仍不把它变成 rectified flow。

原始 latent DiT 的 mean head 有 4 个 epsilon 通道，variance head 另有 4 个通道。官方为精确复现论文结果只 guidance 前三个 epsilon 通道：

$$\hat{\epsilon}^{(k)} = \begin{cases} \epsilon_u^{(k)} + s\big(\epsilon_c^{(k)} - \epsilon_u^{(k)}\big), & k = 1, 2, 3, \\ \epsilon_{c/u}^{(4)}, & k = 4, \end{cases}$$

并且 **所有 variance 通道都不做 guidance**，仍保持各自 conditional/unconditional branch 的值。仓库以 `cfg_channels=first_3` 保存这一复现兼容策略；`cfg_channels=all` 则 guidance 全部 mean channels，是更一般的现代做法。对 3-channel RGB，`first_3` 恰好等于 `all`；对 1-channel MNIST 也没有差别。只有切换到 4-channel latent 时，两者才产生原论文所说的第四通道差异。论文指出全四通道 guidance 也能得到相近结果，因此 first-3 是精确兼容 trick，不是理论必然。

11. Scaling：参数、token 与 Gflops

原始 DiT 通过两个维度增加计算：

- S/B/L/XL：增加 depth、hidden size、heads；
- `/8`、`/4`、`/2`：减小 patch size，增加 token 数。

论文在固定训练框架下观察到更高 forward-pass Gflops 与更好 FID 的经验关系，并报告 DiT-XL/2 在 ImageNet 256 的 FID 2.27。正确表述是“论文设定内的经验 scaling 结果”，不是任何数据和训练预算下参数越大都必然更好。

公平消融至少记录：

$$(N, d, L, h, \#params, \text{Gflops}, \text{steps}, \text{tokens seen}).$$

只固定 epoch 而改变 batch 或数据增强，不是相同训练预算。只比较 parameter count 而忽略 patch 导致的 token 数变化，也不能解释计算来源。

12. 从图像 DiT 到视频 DiT

视频 latent 形状可写为：

$$z \in \mathbb{R}^{B\times C\times F\times H\times W}.$$

若 tube patch 为 $p_t \times p_h \times p_w$，token 数：

$$N = \frac{F}{p_t}\frac{H}{p_h}\frac{W}{p_w}.$$

full space-time attention 成本近似 $O(N^2 d)$。帧数增加会迅速放大成本。

Latte 展示 factorized 思路。令每帧空间 token 数 $S = H_p W_p$，帧 token 数 F_p：

- full attention：$O((F_p S)^2 d)$；
- spatial attention：对每帧做，$O(F_p S^2 d)$；
- temporal attention：对每个空间位置做，$O(S F_p^2 d)$；
- factorized 合计：$O((F_p S^2 + S F_p^2) d)$。

当 $F_p, S > 1$ 时，通常远小于 full attention。代价是空间与时间信息需要跨 block 逐步交换，归纳偏置更强。

后续系统的重点不只 attention：

- CogVideoX：3D VAE、expert adaptive LayerNorm、文本/视频融合；
- Open-Sora：STDiT、任意 shape、数据 curriculum；
- LTX-Video：极高 VAE 压缩、full attention、rectified flow；
- Seedance 1.0：MMDiT、3D/MM-RoPE、多任务条件与级联 refiner；
- HunyuanVideo：dual/single stream、MLLM text encoder、3D VAE、flow、并行与 FP8。

13. 两种 velocity：DDPM v-prediction 不等于 rectified-flow velocity

第 9.1 节的 DDPM v-prediction 使用 variance-preserving Gaussian noising parameterization：

$$x_t = a_t x_0 + \sigma_t \epsilon, \qquad v_{DDPM} = a_t \epsilon - \sigma_t x_0.$$

它只是对同一离散 DDPM state 的 target 重参数化；推理仍可走 $q(x_{t-1} \mid x_t, x_0)$ posterior，仍可使用 learned variance、ancestral DDPM 或 DDIM。

现代视频模型也常说 flow/velocity，但那通常是另一个 probability path 的时间导数。以 rectified flow 的一种线性约定为例：

$$z_t = (1-t) z_0 + t\epsilon, \qquad t \in [0, 1].$$

对 t 求导：

$$\frac{dz_t}{dt} = \epsilon - z_0.$$

训练 velocity field：

$$L_{flow} = \mathbb{E}\left[\| v_\theta(z_t, t, c) - (\epsilon - z_0) \|_2^2\right].$$

注意不同论文可能把时间方向或 target 符号反过来，必须用 path 端点自行求导。推理变为 ODE 数值积分，例如 Euler：

$$z_{t+\Delta t} = z_t + \Delta t \, v_\theta(z_t, t, c).$$

此时不再使用 DDPM 的 $q(x_{t-1} \mid x_t, x_0)$ posterior。DiT backbone 可以保持相似，但 probability path、target 和 sampler 都换了。

对比项	DDPM v-prediction	Rectified-flow velocity
path	$x_t = a_t x_0 + \sigma_t \epsilon$	$z_t = (1-t) z_0 + t\epsilon$
target	$a_t \epsilon - \sigma_t x_0$	$\epsilon - z_0$
数学身份	DDPM state 的旋转参数化	path 对时间的导数
sampler	DDPM posterior / DDIM	ODE solver（如 Euler）
learned DDPM variance	可以保留	不使用该 posterior variance
本仓库 Image DiT	`prediction_type=velocity`	未由这个开关启用

两者偶尔都写作 v，但不能因为变量名相同就互换公式、checkpoint 或 sampler。最可靠的判断方法不是看函数名，而是先写出 path，再问 target 是“旋转后的 denoising target”还是“path 的时间导数”。DDPM v-parameterization 可参考 Progressive Distillation；learned-range variance 与 hybrid loss 来自 Improved DDPM。

14. 代码与公式逐项映射

数学对象	教学代码	核心测试
$q(x_t \mid x_0)$	`DiffusionSchedule.q_sample`	指定 noise 后等于闭式
$\hat{x}_0(x_t, \hat{\epsilon})$	`predict_x0_from_eps`	forward 后精确反演
$v = a_t\epsilon - \sigma_t x_0$	`velocity_target`	v target 与 x_0/ϵ 双向反演
$\log \Sigma_\theta$ learned range	`model_variance`	两端点、raw value 外推
$q(x_{t-1} \mid x_t, x_0)$	`q_posterior`	variance 边界、$t = 0$
MSE + raw/rescaled VB	`diffusion_loss`	stop-gradient、$t = 0$ NLL、$t > 0$ KL
patch projection	`PatchEmbed`	token shape
p_i	`get_2d_sincos_pos_embed`	shape、确定性、位置不同
c_t, c_y	`TimestepEmbedder`、`LabelEmbedder`	odd dim、null token
adaLN-Zero	`DiTBlock`	zero-init block 为 identity
unpatchify	`DiT.unpatchify`	输出空间 shape
EMA	`update_ema`	加权更新与 checkpoint 持久化
CFG	`forward_with_cfg`	$s = 0, 1$、all/first-3、variance 不 guidance
respaced ancestral DDPM	`respaced_schedule`、`sample_loop`	保持选定 $\bar{\alpha}$、映射原 timestep
no-clip reverse loop	`p_sample`、`sample_loop`	finite、$t = 0$、显式 clip policy

15. 建议手推题

1. 从两步 forward process 推出 $q(x_t \mid x_0)$ 的闭式。
2. 对 CIFAR-10 的 $p = 8, 4, 2$ 分别计算 token 数和 attention matrix 元素数。
3. 已知 $d = 384$、heads=6，写出单 head 的 Q, K, V shape。
4. 令六路 adaLN 参数为零，逐行证明 block 是 identity。
5. 从 (x_t, v) 的二维旋转矩阵推导 x_0 与 ϵ 的逆变换，并解释它为何仍属于 DDPM。
6. 写出 learned-range 在 $r_\theta = -1, 1, 3$ 时的 log variance；哪一个是外推?
7. 解释为什么 variance VB 要 detach mean head，以及为什么 $t = 0$ 不能继续使用普通 posterior KL。

8. 证明 respaced beta 的累计乘积等于选定原 timestep 的 $\bar{\alpha}$。
9. 证明 CFG 在 $s = 0, 1$ 的边界，并画出 4-channel latent 的 first-3/all 差异。
10. 对 $F_p = 16, S = 256$ 比较 full 与 factorized attention 的 score pair 数量。
11. 分别写出 DDPM v-prediction 与 $z_t = (1 - t)z_0 + t\epsilon$ 的 flow velocity；若 flow path 改成 $tz_0 + (1 - t)\epsilon$，target 如何变化?

16. 证据边界与延伸阅读

本讲义的 DiT 架构、ImageNet scaling 和官方代码行为来自 DiT 论文 与 官方仓库。learned-range variance、discretized Gaussian decoder NLL、hybrid loss 和 respacing 的背景来自 Improved DDPM 及 DiT 官方仓库中 vendored diffusion code；DDPM v-parameterization 参考 Progressive Distillation。latent diffusion 背景来自 LDM。视频扩展依次参考 Latte、CogVideoX、Open-Sora、LTX-Video、Seedream 2.0、Seedance 1.0、OmniWeaving 与 HunyuanVideo。

Seedance 2.0 当前公开的是 model card。可以引用其明示的输入输出与能力范围，但不能据此断言未披露的数学目标或 block 细节，更不能把 Seedance 1.0 的实现自动当作 2.0 的事实。